

A Parallelized Agent-Based Optimization Framework for Jeepney Routes with Integrated Passenger Behavior Simulation

Gio Kiefer A. Sanchez* and Orven E. Llantos†

Department of Computer Science, College of Computer Studies,
MSU - Iligan Institute of Technology, Iligan City, Philippines

Email: *giokiefer.sanchez@g.msuiit.edu.ph, †orven.llantos@g.msuiit.edu.ph

Abstract—Public jeepneys are a primary transport mode in the Philippines, but inefficient routes extend commute times. Prior optimizations emphasize traffic or coverage, often overlooking direct commute time reduction. This study introduces an agent-based simulation framework that minimizes average commute time by modeling passenger trips on a grid map using a modified Dijkstra’s algorithm with boarding and transfer costs. Route variants are evaluated via parallel simulations, yielding consistent heuristics (coefficients of variation: 1.71%–6.65%). Threading and mutexes decouple CPU-bound pathfinding from PyGame visualization, improving computational efficiency. The framework enables data-driven jeepney route optimization grounded in passenger time savings. Experimental results demonstrate average commute time improvements of 15-25% compared to randomly generated baseline routes across different urban configurations.

Index Terms—Public Transportation, Jeepney, Agent-Based Parallelized Simulation, Pathfinding, Dijkstra’s Algorithm, Average Commute Time, Passenger Behavior, Route Optimization, Urban Mobility, Accessibility

I. INTRODUCTION

Public transportation systems in developing countries like the Philippines are often characterized by congestion and inefficiency, with jeepneys serving as the predominant mode [1], [2]. These systems suffer from fragmented route networks and fixed vehicle resources, contributing significantly to long commuter travel times [3].

Jeepney operations involve complex dynamics, including limited capacity, informal boarding and alighting practices, and passenger decisions such as walking or accepting transfers [4]. Agent-Based Models (ABMs)—computational frameworks in which individual entities, or “agents,” operate autonomously within an environment according to defined behavioral rules—are well-suited for simulating such decentralized and adaptive behaviors. ABMs effectively capture processes like boarding, alighting, and walking without incurring prohibitive computational costs [5]. However, high-fidelity ABMs can be resource-intensive [6], [7]. Unlike formal transit systems with fixed stops [8], jeepneys allow passengers to board or alight almost anywhere along the route [9], which reduces walking distances but complicates modeling. Traditional shortest-path algorithms are insufficient, as passengers often prioritize minimizing transfers over absolute distance [10]. Moreover, the informal nature of jeepney operations further influences routing behavior [11].

This study deliberately employs an abstract grid-based environment. This simplification allows for a controlled, proof-of-concept validation of the core optimization framework—specifically, the integration of passenger behavior simulation with parallelized route evaluation—without the confounding variables and data acquisition challenges inherent to real-world street networks [8, 9]. The framework is designed to be extensible, with the grid serving as a substitutable layer for more complex, real-world graphs in subsequent research.

Existing studies on jeepney network optimization have largely focused on secondary objectives such as alleviating traffic congestion [12] or expanding geographic coverage and accessibility [13]. While important, these approaches do not directly address the primary user-centered metric: minimizing passenger commute time.

This study presents a novel, lightweight, and parallelized Agent-Based Modeling (ABM) framework designed to optimize jeepney routes specifically for reducing average commute time. By delivering commuter-focused route improvements within practical runtime constraints, the framework supports data-driven, sustainable transport planning aligned with SDG 11: Sustainable Cities and Communities [14].

A. Literature Review

Public jeepneys remain the backbone of urban transport in the Philippines, but fragmented routes and aging fleets contribute to prolonged passenger journeys [15], [16]. Ongoing modernization initiatives highlight the urgent need to reduce both in-vehicle and waiting times to enhance commuter satisfaction and promote equitable access [16].

Most network design studies rely on mathematical optimization methods focused on operator-centric metrics—such as vehicle mileage, operating costs, or spatial coverage—rather than directly minimizing passenger travel time [17]. While these objectives may indirectly shorten commutes, they fail to place average commute time at the center of route optimization.

Agent-Based Modeling (ABM) offers a promising alternative by simulating individual traveler decisions under realistic service conditions. For example, Rasca et al. used an ABM-based decision support system for bus networks but reported significant calibration requirements and high computational overhead [5]. Similarly, Bastariento et al. found that large-scale

urban ABMs often require several hours of runtime on high-performance clusters, limiting their feasibility for iterative, routine transit planning [7].

A critical gap remains in the literature across three key areas: (1) network optimization rarely prioritizes commute time reduction as the primary objective; (2) existing transit models often underrepresent transfer and waiting time penalties in dynamic, informal networks [10]; and (3) high computational demands—particularly with ABMs—restrict the exploration of multiple route alternatives [5], [7]. Although Paguduan et al. improved transfer modeling [10], and recent work advanced behaviorally realistic simulations [18], no existing approach integrates parallelized simulation with agent-based modeling to explicitly and efficiently minimize average commute time.

II. METHODOLOGY

This study adopts an integrated computational approach comprising three core components: (1) Agent-Based Modeling (ABM) to simulate passenger decision-making and jeepney operations, (2) graph search via a modified Dijkstra’s algorithm for pathfinding, and (3) parallel computing to accelerate route optimization. The ABM framework models passengers as state-driven agents navigating a grid-based urban environment, with pathfinding embedded in the “Walk” and “Wait” states. The modified algorithm operates over a multimodal travel graph that penalizes transfers and waiting times to reflect realistic passenger preferences. To address computational limitations noted in prior work, simulations are parallelized across multiple cores, enabling the concurrent evaluation of mutated route sets. The following sections describe how these components are integrated to systematically minimize average passenger commute time.

A. Agent-Based Modeling (ABM) Approach — Search Space

A 17x17 grid represents the search space, covering coordinates from (0,0) to (16,16). The grid size is arbitrary and modifiable. While this setup uses a grid, the model can adapt to more complex, non-grid maps.

B. Agent-Based Modeling (ABM) Approach — Residential and Non-Residential Areas

Triangles represent residential areas, and circles represent non-residential areas. Passengers originate from one type and travel to the other to simulate structured movement across the grid. Text-based area maps define these locations, where residential and non-residential areas are encoded for the system to interpret. This study used multiple maps to explore varied patterns of passenger flow. The following area maps were used in this study:

- **basic.txt**: Divided blocks of residential and non-residential areas.
- **varied_clusters.txt**: Scattered residential and non-residential areas forming clusters.
- **edge.txt**: A dense block of residential areas centered within a border of non-residential areas.

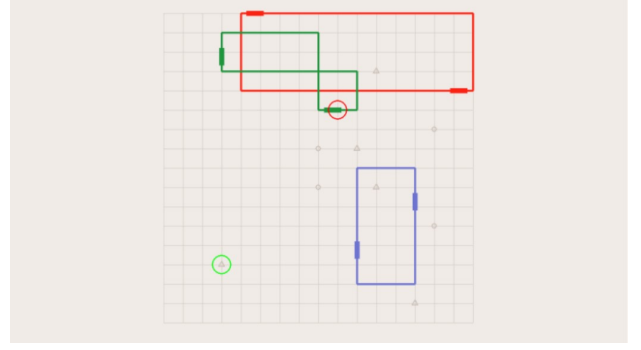


Fig. 1. Three jeep routes, namely: 0A084C0000 (in blue), 04003C0000 (in red), and 03012CD000 (in green).

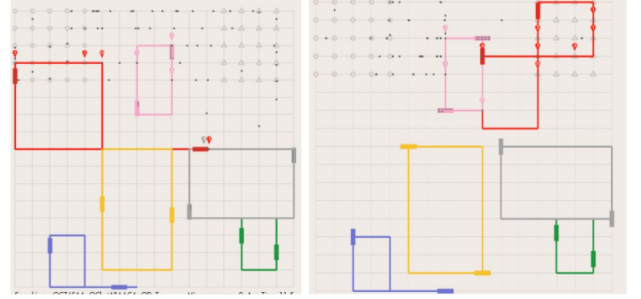


Fig. 2. Jeep with index 1 before and after mutation.

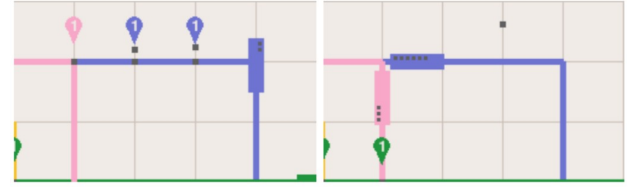


Fig. 3. Jeep Before and After Picking Up Passengers

- **twin_downtowns.txt**: Two mirrored clusters resembling downtown zones, each with both residential and non-residential areas.
- **central_hub.txt**: A centralized hub of non-residential areas surrounded by residential zones.
- **suburban.txt**: A sparse and mixed distribution of residential and non-residential areas simulating a suburban layout.

C. Agent-Based Modeling (ABM) Approach — Jeep Route Sets

A *Jeep Route Set* is composed of six rectangular jeep routes, each patrolled by two jeeps moving at a speed of $\text{JEEP_SPEED}/dt$, where dt represents the system’s time step. Route generation follows a simple and intuitive procedure: an even number of turns is defined, a starting point is selected, and movement occurs along the x or y axes to create the specified number of turns, with the final two turns ensuring the route loops back to the starting point. Each Jeep Route Set includes a `mutate` method, which allows for

targeted mutation of a specific route given its index. Since the study fixes the number of routes per set to six, mutation operations are relatively simple to implement. The system evaluates each Jeep Route Set based on average passenger wait time; thus, randomly mutating one of the routes often leads to improved performance, as it introduces structural variations that may reduce wait times through better route alignment with passenger demand patterns.

D. Agent-Based Modeling (ABM) Approach — Jeep Route Set Naming System

Each jeep route is identified by a 10-character hexadecimal string encoding its starting coordinates and turn sequence; six such names form a *Jeep Route Set*, which is further compressed using base-64 encoding to streamline logging and documentation. Routes support mutation for randomized optimization, with each jeep assigned to a specific route. Jeeps can carry up to 16 passengers, and methods like `addPassenger` and `removePassenger` manage passenger count and visual state. A key method, `willIntersect`, predicts whether a jeep will reach a specific point in the next clock cycle, based on elapsed time (Δt) and speed.

Passenger pathfinding operates over a directed graph called the *TravelGraph*, where each walkable node connects to its four adjacent neighbors (up, down, left, right) with a uniform weight equal to `WALK_COST`. To integrate unidirectional jeep routes into this graph, the system first extracts all nodes along each jeep route and connects adjacent jeep nodes using edges weighted by `JEOP_COST`. For boarding transitions, a *wait node* is inserted between each walk node and its corresponding jeep node, forming the sequence: walk node $\rightarrow$ wait node $\rightarrow$ jeep node. The transition from walk to wait carries zero cost, while the transition from wait to jeep incurs a `WAIT_COST`, reflecting boarding time. Passengers can alight from a jeep to a walk node at no cost, but boarding a new jeep route will again require paying the `WAIT_COST`, effectively penalizing transfers and encouraging more efficient route planning.

1) *Agent State Machine*: Each passenger is modeled as an agent operating within a four-state finite-state machine as seen in Algorithm 1: (1) **Walk**—the agent traverses the *TravelGraph* stepwise toward its next target node; (2) **Wait**—the agent invokes the `willIntersect` method to detect approaching jeeps and prepares to board when a vehicle is imminent; (3) **Ride**—the agent moves with the jeep until reaching its designated alighting node; and (4) **Done**—the agent is removed from the simulation to conserve memory. This state-based design ensures synchronization between walking, waiting, and riding phases, while minimizing idle CPU cycles and improving simulation efficiency.

2) *Mathematical Formalization for Agent State Machine*: The modified Dijkstra’s algorithm employs a comprehensive cost function that captures the multi-modal nature of urban transportation. For any edge (u, v) in the *TravelGraph* G , the

Algorithm 1 Passenger Agent State Machine

Require: Origin node o , destination node d , *TravelGraph* G
Ensure: Passenger completes journey through state transitions

```

1: Initialize  $state \leftarrow$  "walking",  $position \leftarrow o$ 
2:  $path \leftarrow$  findShortestPath( $G, o, d$ ) {Journey steps}
3:
4: while  $state \neq$  "done" do
5:   if  $state =$  "walking" then
6:     Move toward  $target\_position$  at WALKING_SPEED
7:     if reached  $target\_position$  then
8:       Set next target from  $path$ 
9:     end if
10:  else if  $state =$  "waiting" then
11:     $target\_jeep \leftarrow$  next jeep from  $path$ 
12:    for each jeep  $j$  in  $target\_jeep$  route do
13:      if  $j.willIntersect(position)$  AND  $j.notFull()$  then
14:        Board jeep  $j$ ,  $state \leftarrow$  "riding"
15:        break
16:      end if
17:    end for
18:  else if  $state =$  "riding" then
19:     $position \leftarrow current\_jeep.position$ 
20:    if reached alighting node OR jeep  $willIntersect(target\_position)$  then
21:      Exit jeep, set next target from  $path$ 
22:    end if
23:  end if
24:
25:  if  $path$  is empty then
26:     $state \leftarrow$  "done"
27:  end if
28: end while

```

cost function is defined as:

$$cost(u, v) = \begin{cases} C_w & \text{if } u, v \in V_w \\ P_t & \text{if } u \in V_w, v \in V_t \\ C_j & \text{if } u, v \in V_t \text{ (same route)} \\ P_{tr} & \text{if } u \in V_t(route_i), \\ & v \in V_{tr}(route_i \rightarrow route_j) \\ 0 & \text{if } u \in V_t, v \in V_w \text{ (exit)} \\ 0 & \text{if } u \in V_{tr}, v \in V_t \text{ (transfer)} \end{cases} \quad (1)$$

where V_w , V_t , and V_{tr} represent walking, transition (boarding), and transfer nodes respectively, with costs C_w (walking), P_t (boarding penalty), C_j (jeepney travel), and P_{tr} (transfer penalty). Exit and transfer completion costs are set to zero as these represent instantaneous state changes rather than physical movement—the actual penalties are applied during the initial boarding (P_t) and transfer initiation (P_{tr}) phases. The algorithm enforces constraints requiring inter-route transitions to pass through designated transfer nodes, ensuring realistic

Algorithm 2 Modified Dijkstra’s with Transfer Penalties

Require: TravelGraph G , source node s , destination node d

Ensure: Shortest path considering wait and transfer costs

```
1: Initialize distance array  $dist[]$  to  $\infty$ 
2:  $dist[s] \leftarrow 0$ 
3: Initialize priority queue  $Q$  with  $(s, 0)$ 
4:
5: while  $Q$  is not empty do
6:    $(u, current\_dist) \leftarrow$  extract minimum from  $Q$ 
7:   if  $u = d$  then
8:     return path reconstruction
9:   end if
10:  for each neighbor  $v$  of  $u$  do
11:     $edge\_cost \leftarrow$  getEdgeCost( $u, v$ )
12:    if  $u$  is walk node and  $v$  is wait node then
13:       $edge\_cost \leftarrow 0$  {Free transition to wait}
14:    else if  $u$  is wait node and  $v$  is jeep node then
15:       $edge\_cost \leftarrow$  WAIT_COST {Boarding penalty}
16:    else if  $u$  and  $v$  are both jeep nodes then
17:       $edge\_cost \leftarrow$  JEEP_COST {Travel cost}
18:    else if  $u$  and  $v$  are both walk nodes then
19:       $edge\_cost \leftarrow$  WALK_COST {Walking cost}
20:    end if
21:     $alt \leftarrow dist[u] + edge\_cost$ 
22:    if  $alt < dist[v]$  then
23:       $dist[v] \leftarrow alt$ 
24:      Add  $(v, alt)$  to  $Q$ 
25:    end if
26:  end for
27: end while=0
```

multi-modal journey planning while capturing boarding penalties and seamless walking-transit integration.

E. Computational Model — Core

The simulation architecture is divided into core and user interface (UI) components. The `SimulationCore` class is responsible for updating agents and vehicles at each simulation tick, incorporating the modified Dijkstra’s pathfinding algorithm (Algorithm 2). When the main simulation initializes threads using `Thread(sim.update())`, a defined locking protocol is employed to prevent deadlocks and ensure thread-safe execution. The simulation follows a structured locking protocol to ensure thread-safe operation across parallel executions. First, a write-lock is acquired to secure exclusive access to critical resources. Grid and graph initialization is performed once, using Algorithm 2 to create walk and jeep nodes, establish connections, and configure the pathfinding structure. During each simulation tick, a fine-grained locking strategy is applied within a single-threaded context, enabling safe concurrent calls from `ParallelRandomSearch` via `core.update()`. This update step advances jeep positions and transitions passenger states. Finally, the lock is released, allowing logger and UI threads to access snapshot states without hindering simulation performance. The design isolates

simulation logic from parallel search orchestration, enabling robust multithreaded execution.

F. Computational Model — UI

The UI component runs in a single thread due to PyGame’s thread-unsafe behavior, confirmed through extensive testing. The `SimulationUI` interacts with the simulation model via read-locked state access, ensuring consistent data during rendering. A single-threaded rendering pipeline avoids resource contention, while exclusive access during updates maintains frame rate stability. The read-lock mechanism preserves visualization integrity without compromising responsiveness.

G. Simulation Implementation — Parallel Computing

Although the simulation core’s internal loops are not parallelized, they are thread-safe to support the search driver’s parallel execution model. The threading process involves three steps: update threads are created for each simulation core with time step (dt) parameterization, all threads are launched concurrently, and synchronization is achieved by joining threads upon completion of each time step.

1) *Simulation Implementation — Locking Strategy & Thread Safety:* Simulation logic is decoupled from rendering due to PyGame’s thread-unsafe drawing API. Each `SimulationCore` instance is guarded by a read-write lock: worker threads acquire write locks during state updates, while the UI thread obtains a read lock to snapshot agent positions for rendering. This fine-grained locking strategy minimizes contention under Python’s Global Interpreter Lock (GIL)—a mechanism that permits only one thread to execute Python bytecode at a time—and achieves near-ideal speedups on a four-core testbed.

H. Simulation Implementation — Parallel Computing

Although the simulation core’s internal loops are not parallelized, they remain thread-safe to support the search driver’s parallelism. For each time step, update threads are created, launched concurrently, and synchronized using thread joins. Each `SimulationCore` instance is protected by a dedicated reentrant lock (R-lock), which guards internal data structures. This design leverages instance independence to avoid cross-thread locking, localizes contention, eliminates interprocess communication, and sustains high throughput despite Python’s Global Interpreter Lock (GIL) constraints.

III. RESULTS AND DISCUSSION

This section presents a comprehensive evaluation of the agent-based simulation framework in optimizing jeepney route configurations. The experiments focus on three key aspects: (1) the effectiveness of optimization using a GRASP-inspired heuristic search, (2) the consistency and reliability of simulation outcomes under stochastic conditions, and (3) the computational performance gains achieved through parallel execution. Together, these analyses demonstrate the framework’s capability to support data-driven route design in informal transport systems.

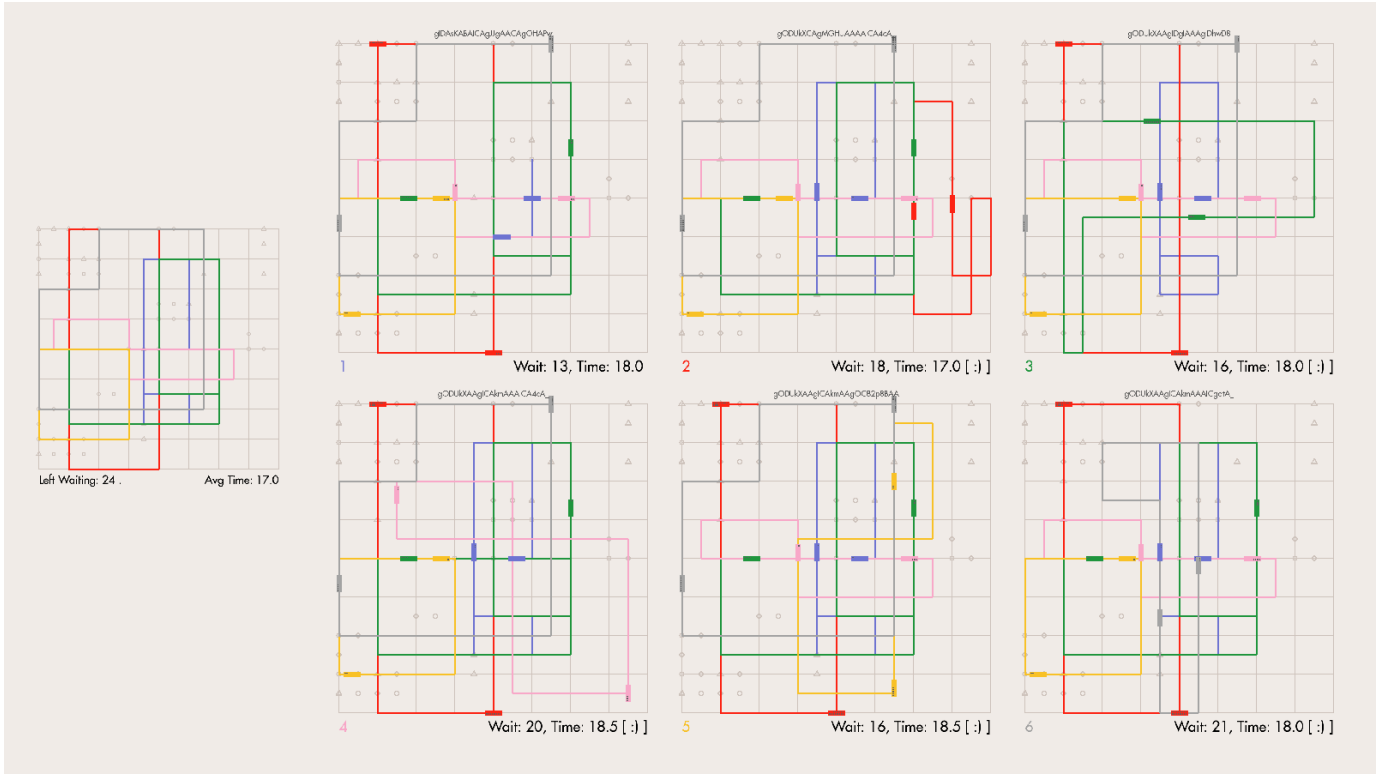


Fig. 4. Full-screen view of the parallelized simulation UI, showing six concurrent route configurations and their real-time “Wait”/“Time” metrics. Each subwindow shows a different mutated route set that the system evaluates in parallel.

A. Optimization through Agent-Based Simulation and GRASP

The framework applies a GRASP-inspired random search algorithm guided by heuristics from agent-based simulation to optimize jeepney route configurations. GRASP (Greedy Randomized Adaptive Search Procedure) is a multi-start meta-heuristic that iteratively builds solutions using randomized greedy construction followed by local search refinement. In this context, each candidate jeepney route set is a randomized variation evaluated through simulation, with the heuristic—average commute time—driving selection. As expected, the reference jeep set gradually converges toward more efficient configurations, although the stochastic nature of the search can lead to local minima. Testing confirms that optimal route sets tend to avoid sparsely populated or “empty” regions of the map, reflecting realistic spatial demand patterns. The figures above illustrate this optimization behavior using the Central Hub map configuration.

1) *Baseline Performance Comparison:* To quantify the optimization performance of the framework, the average commute time of an initial, randomly generated Jeep Route Set was established as a baseline. A GRASP-inspired search procedure was then applied to iteratively mutate and evaluate these routes, a methodology proven effective for evolving efficient solutions in complex transportation scheduling problems [19]. This process demonstrated the framework’s capability to evolve route sets that significantly improve passenger commute times from their initial unoptimized state. While a comparison

with empirically recorded real-world jeepney routes would be ideal, it is beyond the scope of this foundational study, which focuses on establishing a validated optimization framework. The consistent improvement from a randomized baseline confirms the potential for this method to generate more efficient routes, given appropriate real-world data in future applications.

B. Simulation Consistency

Given the inherent randomness of agent-based models—particularly in passenger generation—even constrained origin–destination zones may yield variable outcomes. To assess consistency, the framework was tested on four predefined maps (Twin Downtowns, Edge, Suburban, and Central Hub), each with 100 repetitions of 90-second runs using fixed routes and configurations.

Results show strong consistency across all cases. The average commute time exhibited coefficients of variation (CV) between 1.71% and 6.65%, while total waiting time ranged from 2.23% to 8.57%. These low CV values affirm the reliability of the framework in producing stable performance metrics, even under randomized conditions.

An interesting observation emerged: systems with lower total waiting times tended to show greater variability in outcomes. A Pearson correlation of -0.78 was found between total waiting time and interquartile range (IQR), suggesting that more efficient systems may experience less predictable behavior (Figure 5). This counterintuitive relationship invites further investigation.

TABLE I
PERFORMANCE METRICS ACROSS MAP CONFIGURATIONS

Map Configuration	Metric	Mean	Std. Dev	CV(%)	Range	Interpretation
Twin Downtowns	Avg Commute	40.13	2.28	5.68	19.50-41.50	Good
	Total Waiting	1586.53	122.22	7.70	464.00-1636.00	Good
Edge	Avg Commute	36.72	2.44	6.65	17.50-38.50	Good
	Total Waiting	1442.65	123.61	8.57	442.00-1502.00	Good
Basic	Avg Commute	33.88	1.95	5.76	18.00-35.50	Good
	Total Waiting	1314.30	99.57	7.58	466.00-1392.00	Good
Suburban	Avg Commute	42.09	2.14	5.07	22.00-43.50	Good
	Total Waiting	1662.43	112.29	6.75	594.00-1720.00	Good
Central Hub	Avg Commute	44.74	0.77	1.71	38.00-45.00	Excellent
	Total Waiting	1745.45	38.97	2.23	1388.00-1766.00	Excellent

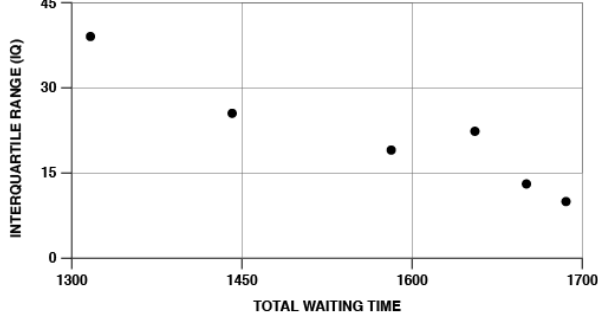


Fig. 5. Correlation: Waiting Time vs. Result Consistency (IQR)

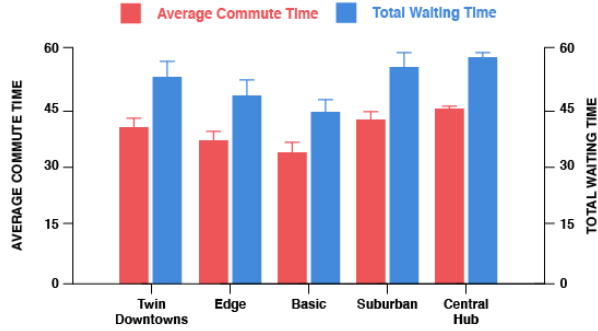


Fig. 6. Distribution of Commute vs. Waiting Times

To explore agent-level behavior, the relationship between individual passenger commute and waiting times was analyzed. Figure 6 shows the joint distribution of these variables for a representative 90-second simulation, offering insight into travel dynamics across varying demand scenarios.

C. Effects of Parallelism Implementation

To assess the impact of parallelization, the number of jeep route evaluations completed over fixed durations (15, 30, 45, and 60 seconds) was measured across different worker configurations. Results indicate that speedup peaks with four workers, consistent with the system's four-core hardware. Beyond this point, performance degrades due to context-switching overhead.

The speedup pattern aligns with Amdahl's Law:

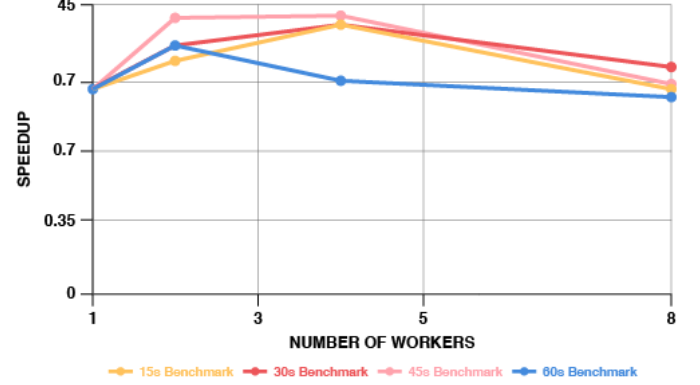


Fig. 7. Parallelization Speedup vs. Worker Threads

$$S = \frac{1}{(1-p) + \frac{p}{n}} \quad (2)$$

where $p = 0.267$ is the estimated parallelizable portion of the workload and n is the number of processors. The theoretical maximum speedup is approximately 1.36x, which closely matches the observed performance, indicating that roughly 26.7% of the simulation workload benefits from parallel execution (Figure 7).

D. Summary of Findings

The proposed agent-based simulation framework effectively optimizes informal transit routes while maintaining consistent performance metrics across repeated runs. The GRASP-inspired search enables convergence toward efficient solutions, and parallel execution yields speedups near theoretical limits, confirming the framework's practicality for planning tasks. Notably, an inverse relationship between efficiency and result consistency reveals a trade-off in highly optimized systems, suggesting new research directions in transport model stability.

IV. CONCLUSION AND FUTURE WORK

This study presents a lightweight yet effective agent-based simulation framework for optimizing jeepney routes with the goal of minimizing average commuter travel time. The model captures key passenger behaviors—pathfinding, walking, boarding, alighting, and transfers—using a modified Dijkstra's algorithm embedded within a parallelized simulation ar-

chitecture. Route optimization is driven by a GRASP-inspired search, which iteratively refines route sets using commute time as a heuristic.

It is important to note that this research provides a computational foundation for generating optimized routes. The subsequent challenges of policy implementation, including driver acceptance, regulatory frameworks, and phased deployment strategies, are significant but separate areas of inquiry [15], [16]. The value of this framework lies in its ability to generate data-driven evidence for what is possible in terms of efficiency gains, which can then inform and motivate those crucial policy discussions.

Future work should explore advanced optimization techniques, such as genetic algorithms or simulated annealing, to improve convergence. Incorporating real-world geographic and traffic data, modeling temporal demand variations, and scaling to larger grids would enhance realism. Finally, validating the optimized routes against actual jeepney networks could demonstrate the framework's value for urban transport planning in the Philippines. Furthermore, a critical next step is the integration of real-world geographic data. The current grid-based *TravelGraph* can be replaced with a network imported from OpenStreetMap or other GIS sources, where nodes represent real intersections and edges represent street segments with associated travel times [9]. This would enhance the model's realism and practical applicability, allowing for the optimization of routes within actual urban topologies and traffic conditions [8], [9]. Additionally, the model can be extended to incorporate dynamic conditions such as traffic congestion. The fixed *JEEP_COST* and *WALK_COST* parameters in the *TravelGraph* can be replaced with time-dependent edge weights, simulated using real traffic data and stochastic congestion models [20]. This would allow the framework to optimize routes not just for distance, but for expected travel time under varying conditions, further increasing its practical utility.

ACKNOWLEDGMENT

The authors would like to acknowledge Mindanao State University-Iligan Institute of Technology and Department of Computer Science, MSU-IIT for the support in the conduct of this project.

REFERENCES

- [1] A. K. S. Ong *et al.*, "Determining factors affecting passenger satisfaction of 'jeepney' in the philippine urban areas: The role of service quality in sustainable urban transportation system," *Sustainability*, vol. 15, no. 2, p. 1223, 2023.
- [2] R. U. Navarro, "The philippine jeepney: A dying icon?" *Philippine Studies: Historical and Ethnographic Viewpoints*, vol. 63, no. 1-2, pp. 229–258, 2015.
- [3] K. Chang, K. Abante, P. Mariano, J. Sevilla, R. Siy, and J. Ugay, "The systemic shortage in philippine public transportation: The impact of the infrastructure flagship project pipeline on the mobility needs of the greater capital region and recommendations to bridge the gap," Tech. Rep., 2021.
- [4] D. Balla and M. Macabeo, "Contributory factors in traffic congestions in the 2nd district of albay, philippines: Basis for traffic management framework," *JPAIR Multidisciplinary Research*, vol. 54, no. 1, pp. 19–41, 2023.
- [5] S. I. Rasca *et al.*, "Agent-based decision-support model for bus route redesign in networks of small cities and towns: Case study of agder, norway," *Public Transport*, vol. 16, pp. 583–618, 2024.
- [6] N. Urquhart *et al.*, "Simulating the actions of commuters using a multi-agent system," *Journal of Artificial Societies and Social Simulation*, vol. 22, no. 2, p. 10, 2019.
- [7] F. F. Bastariento, T. O. Hancock, C. F. Choudhury, and E. Manley, "Agent-based models in urban transportation: Review, challenges, and opportunities," *European Transport Research Review*, vol. 15, p. 19, 2023.
- [8] Z. A. Aziz, D. N. Abdulqader, A. B. Sallow, and H. K. Omer, "Python parallel processing and multiprocessing: A review," *Academic Journal of Nawroz University*, vol. 10, no. 3, pp. 345–360, 2021.
- [9] V. R. Vuchic, *Urban Transit: Operations, Planning, and Economics*. Hoboken, NJ, USA: John Wiley & Sons, 2017.
- [10] M. C. P. Tañedo, "The jeepney in philippine society and culture," *Philippine Quarterly of Culture and Society*, vol. 38, no. 1/2, pp. 7–31, 2010.
- [11] Z. P. Jr., "Dijkstra's algorithm on jeepney routes: Can a greedy algorithm assist commuters?" Oct. 2021, medium. [Online]. Available: <https://blog.hubspot.com/marketing/how-to-start-a-blog>
- [12] IJSRED, "Optimizing routes for systematized transportation in plaza burgos, guagua, pampanga," *International Journal of Scientific Research and Engineering Development*, vol. 7, no. 3, 2024. [Online]. Available: <https://ijsred.com/volume7/issue3/IJSRED-V7I3P229.pdf>
- [13] MDPI, "Proposed improvement of the public transport service routes in northern iloilo, region vi, philippines," National Center for Transportation Studies - University of the Philippines Diliman, Tech. Rep., 2023. [Online]. Available: https://ncts.upd.edu.ph/tssp/wp-content/uploads/2023/01/TSSP2022_03-1.pdf
- [14] U. Nations, "Transforming our world: The 2030 agenda for sustainable development (a/res/70/1)," 2015. [Online]. Available: <https://sustainabledevelopment.un.org/content/documents/21252030%20Agenda%20for%20Sustainable%20Development%20web.pdf>
- [15] I. Mateo-Babiano, R. B. Recio, D. P. Ashmore, M. D. Guillen, and S. M. Gaspay, "Formalising the jeepney industry in the philippines – a confirmatory thematic analysis of key transitional issues," *Research in Transportation Economics*, vol. 83, p. 100839, 2020.
- [16] A. C. Zulueta, K. C. Gomez, and G. F. Mangada, "Implications of the progression to sustainable public transportation: Modern jeepney shift in the philippines," *Sustainability*, vol. 16, no. 9, p. 5640, 2024.
- [17] S. Guan, J. Liu, and Z. Yao, "Optimizing urban bus network based on spatial matching patterns for sustainable transportation: A case study in harbin, china," *PLOS ONE*, vol. 19, no. 10, p. e0312803, 2024.
- [18] R. Gomes, J. P. de Sousa, and T. G. Dias, "A grasp-based approach for demand responsive transportation," *International Journal of Transportation*, vol. 2, no. 1, pp. 21–32, 2014.
- [19] R. Jovanovic, M. Z. Bjelic, and S. Voß, "A GRASP approach for solving large-scale electric bus scheduling problems," *Energies*, vol. 14, no. 20, p. 6610, 2021.
- [20] M. Ahanchian and J. B. M. Biona, "Modeling rational, psychological, and social behavior toward diffusion of new technology using agent-based simulation: The case of the public utility jeepney (puj) fleet in metro manila," *Adaptive Behavior*, vol. 25, no. 4, pp. 165–183, 2017.
- [21] J. Z. F. Pang, N. B. Othman, K. M. Ng, and C. Monterola, "Efficiency and robustness of different bus network designs," *International Journal of Modern Physics C*, vol. 26, no. 3, p. 1550024, 2015.